

# Cryptography Fundamentals and Pentesting Basics

## [Python code solution with GitHub](#)

### 1. Caesar Cipher:

Task: decrypt given cyphertext: "Hvs Eiwqy Pfckb Tcl Xiadg Cjsf Hvs Zonm Rcu."

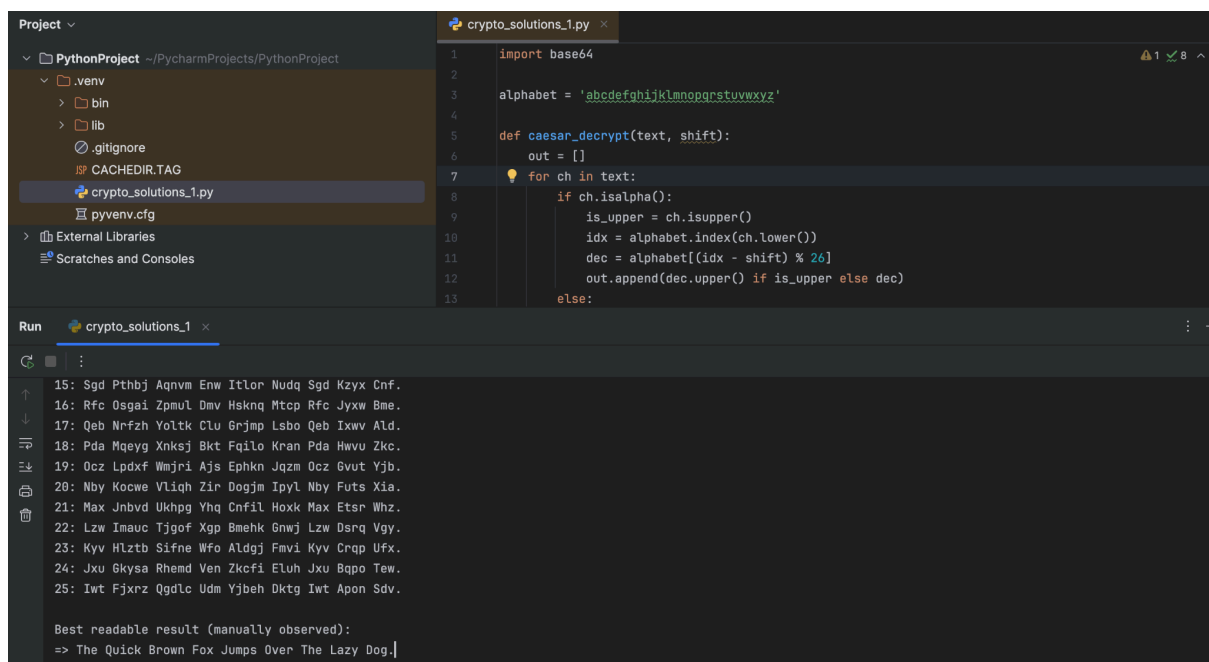
- Challenge: Perform Frequency Analysis or brute-force attack to decrypt a ciphertext. Provide python code solution with GitHub.
- Discussion: Why is Caesar cipher insecure? Where might legacy systems still use similar encryption?

**Decrypted plaintext:**

**The Quick Brown Fox Jumps Over The Lazy Dog.**

**Why it's insecure** - Only 26 possible shifts, Simple inspection reveals plaintext quickly and offers no resistance against known-plaintext.

**Where similar/simple schemes still appear** - Educational examples and puzzles for teaching, very old devices or software might use XOR with a fixed key or Caesar-like substitutions as a legacy "protection".



```
1 import base64
2
3 alphabet = 'abcdefghijklmnopqrstuvwxyz'
4
5 def caesar_decrypt(text, shift):
6     out = []
7     for ch in text:
8         if ch.isalpha():
9             is_upper = ch.isupper()
10            idx = alphabet.index(ch.lower())
11            dec = alphabet[(idx - shift) % 26]
12            out.append(dec.upper() if is_upper else dec)
13        else:
14            out.append(ch)
15
16 # Example usage
17 cyphertext = "Hvs Eiwqy Pfckb Tcl Xiadg Cjsf Hvs Zonm Rcu."
18 shift = 13
19 plaintext = caesar_decrypt(cyphertext, shift)
20 print(plaintext)
```

Run crypto\_solutions\_1

```
15: Sgd Pthbj Aqnmv Enw Itlor Nudq Sgd Kzyx Cnf.
16: Rfc Osgai Zpmul Dmv Hsknq Mtcp Rfc Jyxw Bme.
17: Qeb Nrfzh Yoltk Clu Grjmp Lsbo Qeb Ixwv Ald.
18: Pda Mgeyg Xnksj Bkt Fqilo Kran Pda Hwvu Zkc.
19: Qcz Lpdxr Wmjri Ajs Ephkm Jqzm Qcz Gvut Yjb.
20: Nby Kocwe Vliqh Zir Dogjm Ipyl Nby Futs Xia.
21: Max Jnbvd Ukhpg Yhq Cnfil Hoxk Max Etsr Whz.
22: Lzw Imauc Tjgof Xgp Bmehk Gnwj Lzw Dsrq Vgy.
23: Kyv Hlztb Sifne Wfo ALdgg Fmvi Kyv Crqp Ufx.
24: Jxu Gkysa Rhemd Ven Zkcfi Eluh Jxu Bqpo Tew.
25: Iwt Fjxrz QgdLc Udm Yjbeh Dktg Iwt Apon Sdv.

Best readable result (manually observed):
=> The Quick Brown Fox Jumps Over The Lazy Dog.
```

## 2. XOR Encryption/Decryption:

### Step 1: Caesar Cipher Challenge

- Ciphertext: mznxpz
- Challenge: Perform a brute-force or frequency analysis attack to decrypt the Caesar-encrypted text.
- Clue: The ciphertext is an encrypted anagram of the passphrase.

Ciphertext: **mznxpz**

Brute-force result (shift = 21): **rescue**

### Step 2: Solve the Anagram

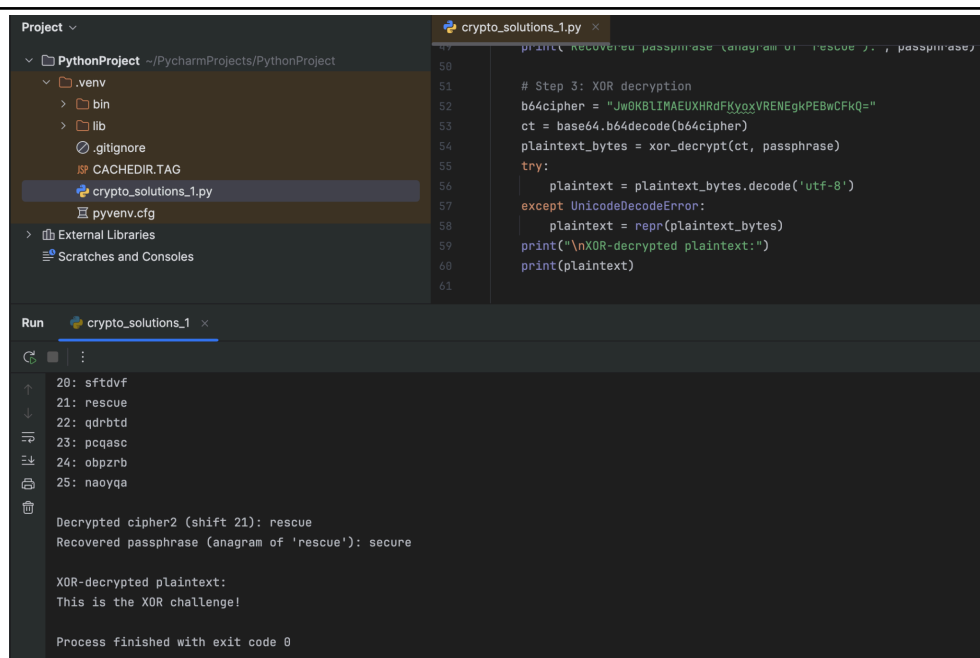
- Decrypted Text: Rearrange the decrypted words to form the original passphrase.
- Hint: The final passphrase is a fundamental concept in cryptography.

Anagram of **rescue** → passphrase: **secure**

### Step 3: XOR Decryption

- Given Ciphertext in base64: Jw0KBIIMAEUXHRdFKyoxVRENEgkPEBwCFkQ=
- Instructions: Use the recovered passphrase from Step 2 to XOR-decrypt the message (first you must convert base64 then decrypt).

Base64 ciphertext: **Jw0KBIIMAEUXHRdFKyoxVRENEgkPEBwCFkQ=**  
Base64 → bytes, XOR key = **secure** → Decrypted plaintext: **This is the XOR challenge!**



```
Project
└─ PythonProject
   └─ .venv
      ├── bin
      ├── lib
      ├── .gitignore
      ├── CACHEDIR.TAG
      ├── crypto_solutions_1.py
      └── pyvenv.cfg
  └─ External Libraries
     └─ Scratches and Consoles

crypto_solutions_1.py
47 print(RECOVERED passphrase (anagram of 'rescue'): , passphrase)
48
49 # Step 3: XOR decryption
50 b64cipher = "Jw0KBIIMAEUXHRdFKyoxVRENEgkPEBwCFkQ="
51 ct = base64.b64decode(b64cipher)
52 plaintext_bytes = xor_decrypt(ct, passphrase)
53 try:
54     plaintext = plaintext_bytes.decode('utf-8')
55 except UnicodeDecodeError:
56     plaintext = repr(plaintext_bytes)
57 print("\nXOR-decrypted plaintext:")
58 print(plaintext)
59
60
61

Run
crypto_solutions_1
20: sftdvf
21: rescue
22: qdrbtd
23: pcqasc
24: obpzrb
25: naoyqa

Decrypted cipher2 (shift 21): rescue
Recovered passphrase (anagram of 'rescue'): secure

XOR-decrypted plaintext:
This is the XOR challenge!

Process finished with exit code 0
```